



Latent-DARM: Bridging Discrete Diffusion and Autoregressive Models for Reasoning

弥合离散扩散模型与自回归模型的推理鸿沟

LATENT-DARM: BRIDGING DISCRETE DIFFUSION
AND AUTOREGRESSIVE MODELS FOR REASONING

Lina Berrayana^{1†‡}, Ahmed Heakl^{2†}, Muhammad Abdullah Sohail², Thomas Hofmann³,
Salman Khan², Wei Chen^{4‡}

¹EPFL

²MBZUAI

³ETH Zürich

⁴Microsoft Research Asia

李宏扬
2026.4.1

研究动机



- 现有问题：
- 多智能体系统主要依赖自回归模型（ARM），生成过程是**严格顺序**的
- 离散扩散模型（DDLM）支持**非顺序、可全局修改**的生成，在复杂推理和规划任务中表现更优
- 但DDLM存在**文本流畅性不足**的问题，限制了与ARM的有效协作

背景知识



- 自回归模型：
- 通过从左到右、逐词预测的方式生成文本，每一步只依赖之前已经生成的词。
- 以论文中的数学问题为例
- 问题： "一个长方形的长是宽的3倍，周长是48米。求面积？ "
- ARM的生成步骤（问题的内容会作为输入的初始化）

步骤	当前输入	预测下一个词	输出序列
1	[START]	"设"	["设"]
2	["设"]	"宽"	["设", "宽"]
3	["设", "宽"]	"为"	["设", "宽", "为"]
4	["设", "宽", "为"]	"x"	["设", "宽", "为", "x"]
5	["设", "宽", "为", "x"]	", "	["设", "宽", "为", "x", ", "]



背景知识

- Step 1: "设" → 无法预知后面要设什么
- Step 2: "设宽" → 确定了设宽
- Step 3: "设宽为" → --
- Step 4: "设宽为 x " → --
- Step 5: "设宽为 x , " → --
- Step 6: "设宽为 x , 则" → --
- Step 7: "设宽为 x , 则长" → 确定了下一步设长
- Step 8: "设宽为 x , 则长为" → --
- Step 9: "设宽为 x , 则长为 $3x$ " → --
- 如果中间某步错了, 比如:
- Step 4: "设宽为 y " → 后面所有推理都基于 y
- 但到Step 9才发现应该是" $3x$ ", 但已经无法修改了!

ARM (自回归模型)

缺乏对全局的观察能力

并且只能“往前看”不能“往回看”

背景知识

- DDLM(离散扩散语言模型)

通过迭代去噪过程生成文本的语言模型。与自回归模型从左到右逐词生成不同，DDLM从一个完全被掩码的序列开始，通过多个步骤逐步揭示出完整的文本。

第1步（初始化）：[M] [M] [M] [M] [M] [M] [M] [M] ← 全部是掩码

第2步（第一次去噪）：[M] [M] [M] [M] [M] "x" [M] [M] ← 先揭示关键信息

第3步（第二次去噪）：[M] "宽" [M] "x" [M] "长" [M] "3x" ← 揭示更多内容

第4步（第三次去噪）："设" "宽" "为" "x" "则" "长" "为" "3x" ← 完整序列

第5步（最终输出）："设宽为x，则长为3x"

背景知识

- DDLM关键特性：
- 可以任意顺序揭示token（不是从左到右）
- 可以同时揭示多个token（并行解码）
- 可以在后续步骤修正之前的选择

相对于ARM的核心优势：

1. 全局规划能力
2. 推理速度快（算力充足时可以并行揭示token）





背景知识

DDLMM的短板：流畅性问题

"DDLMMs still lag behind ARMs in terms of text fluency"

冗余、重复性输出

"设宽为 x ，设宽为 x ，那么长为 $3x$ ，长是宽的3倍，所以长= $3x$ ，周长公式是 $2 \times (\text{长} + \text{宽}) = 48 \dots$ "

- 重复 ("设宽为 x "出现两次)
- 冗余表达
- 逻辑跳跃

文章动机

“尽管取得了这些进展，现有的大多数多智能体系统仍然完全依赖自回归语言模型（ARM），它们以严格的顺序方式逐词生成输出。其结果是，智能体工作流的每个阶段都本质上是自回归的。”

“因此，完全依赖ARM可能会限制多智能体系统的全部潜力，特别是在需要灵活规划或全局推理的任务上。”

例如：

问题

A比B多5，B比C少3，求A比C多多少”

ARM可能学到的“捷径”

看到“多”就加，看到“少”就减
→ $A+5-3=C$

真实需要的规划

需要建立A、B、C的关系链

背景知识

数据1: ARM在复杂任务上性能骤降 (表1)

任务难度	Llama-3.2-3B (ARM only)	说明
DART-1 (较简单)	44.5%	尚可
DART-2	35.5%	下降
DART-3	28.0%	显著下降
DART-4	34.5%	波动
DART-5 (最难)	36.5%	低水平

随着任务复杂度增加，ARM性能没有稳定提升，甚至在中间难度出现下降。

数据2: AIME 2024上的惨淡表现 (表1)

模型	AIME 2024准确率
Llama-3.2-3B (ARM only)	3.5%
Qwen2.5-7B (ARM only)	2.5%

规划——执行



由于DDLMM拥有更好的全局洞察力与反思能力，而ARM能更好完成单一直观任务
于是DDLMM规划——ARM执行 这一agent行为链条自然地会被想到



但是 实验表明，直接的DDLMM->ARM或ARM->DDLMM通常会导致任务直接失败

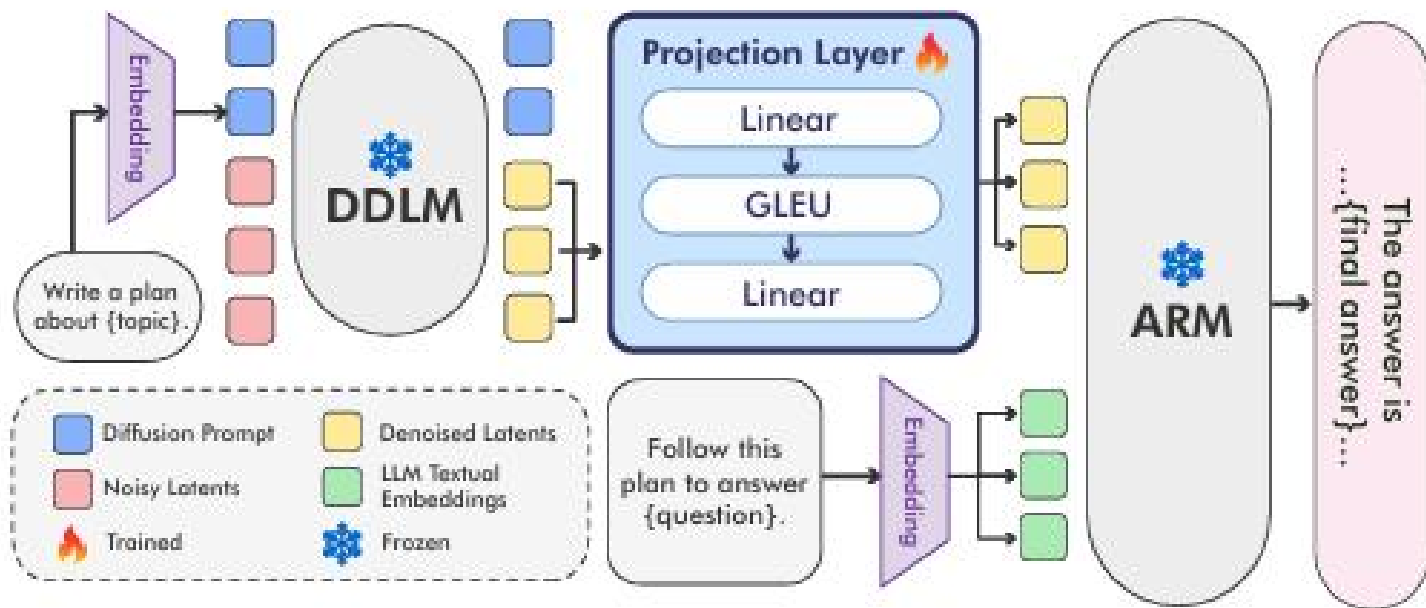
DDLMM → DDLMM 成功，但 DDLMM → ARM 失败

这说明：不是DDLMM的计划本身差，而是ARM无法理解DDLMM的文本输出

隐空间投影

- 原始方法：输入问题->DDL_M编码->DDL_M处理->DDL_M解码->ARM编码->ARM处理->ARM解码

作者提出：跳过将DDL_M的输出翻译为文本再喂给ARM这个过程，直接在向量组层面完成两部分的嵌合



隐空间投影

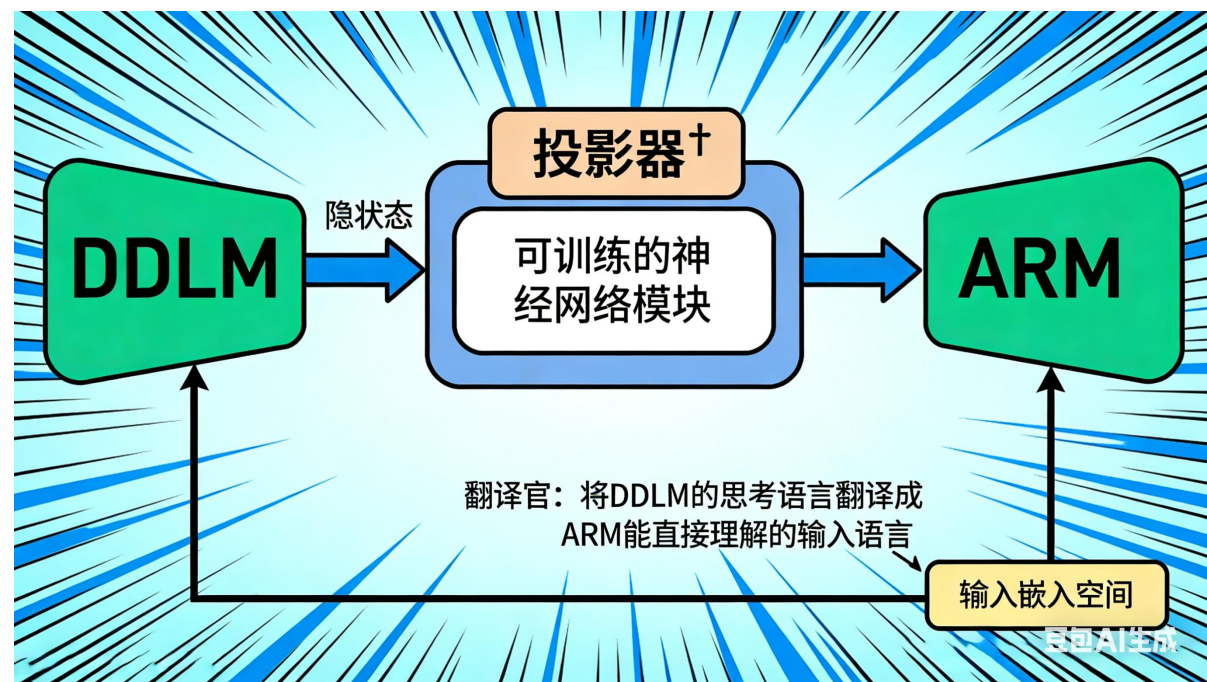


- 投影器 f 是一个可训练的神经网络模块，负责将DDLMM的隐状态映射到ARM的输入嵌入空间。
- 核心作用：

充当"翻译官"：把DDLMM的"思考语言"翻译成ARM能直接理解的"输入语言"

为什么需要投影器？

问题	说明
维度不匹配	DDLMM: 4096维 ARM:1024维
空间不匹配	双向训练 vs 单向训练
语义不对齐	同一概念在不同模型中表示不同



投影器结构



```
class LatentProjector(nn.Module):
```

```
    def __init__(self, d_ddlm=4096, d_arm=1024):
```

```
        super().__init__()
```

```
        self.fc1 = nn.Linear(d_ddlm, 2048)    # 降维
```

```
        self.act1 = nn.GELU()                 # 非线性
```

```
        self.fc2 = nn.Linear(2048, d_arm)      # 降维到目标维度
```

```
        self.act2 = nn.GELU()                 # 非线性
```

```
        self.fc3 = nn.Linear(d_arm, d_arm)     # 特征精炼
```

```
        self.norm = nn.RMSNorm(d_arm)         # 归一化
```

可训练

训练方法



训练目标 (论文公式5)

$$\min_{\theta} \mathbb{E}_{(q,a) \sim \mathcal{D}} [-\log p_{\text{ARM}}(a \mid f_{\theta}(h_{\text{DDL}}(q)), q)]$$

对每一组训练样本 (q, a) , 训练目标是 找到一个最好的投影器参数 θ 使其在同一DDL处理后的 q 经ARM处理结束后与 a 相比 似然损失最小

Algorithm 1 Latent-DARM Training

Input: data $\mathcal{D}$, planner θ_{DDL} , executor θ_{ARM} , projection f_{θ} , rate η , epochs E , size B

- 1: Initialize θ randomly
- 2: **for** each epoch **do**
- 3: **for** each batch **do**
- 4: **for** i in batch **do**
- 5: $h_i^{\text{DDL}} = \text{DDL}_{\text{planner}}(q_i)$
- 6: $h_i^{\text{proj}} = f_{\theta}(h_i^{\text{DDL}})$
- 7: $h_i^{\text{ARM}} = [h_i^{\text{proj}}; \text{embed}_{\text{ARM}}(q_i)]$
- 8: $\mathcal{L} = -\frac{1}{B} \sum_{i=1}^B \log p_{\text{ARM}}(a_i \mid h_i^{\text{ARM}})$
- 9: $\theta = \theta - \eta \nabla_{\theta} \mathcal{L}$

实验结果

Model / Setting	ARC-E	ARC-C	DART-1	DART-2	DART-3	DART-4	DART-5	AIME24	MMLU
Accuracy									
DeepSeek-R1 (ARM only)	94.0	88.0	89.0	81.5	85.5	75.0	61.5	28.5	60.0
Qwen3-1.7B (ARM only)	92.0	86.0	89.5	86.0	83.0	73.0	49.0	8.5	68.5
Llama-3.2-3B (ARM only)	87.5	79.5	44.5	35.5	28.0	34.5	36.5	3.5	54.0
LLaDA-8B → Llama-3.2-3B (text-space)	90.5	82.5	53.5	43.0	35.5	30.0	27.0	0.0	52.5
LLaDA-8B → Llama-3.2-3B (latent-space)									
64 tokens plan	85.0	78.5	78.5	62.5	57.0	63.0	54.0	12.5	52.0
128 tokens plan	87.5	76.5	70.5	43.0	43.0	49.0	36.0	14.0	44.0
256 tokens plan	85.0	81.0	70.0	62.0	50.0	62.0	52.5	14.0	15.5
Number of Tokens in average									
DeepSeek-R1 (ARM only)	398	504	1420	1833	2076	2418	3068	3832	760
Qwen3-1.7B (ARM only)	282	397	1024	1669	2112	2550	3106	4036	984
Llama-3.2-3B (ARM only)	4	4	17	16	22	28	41	462	4
LLaDA-8B → Llama-3.2-3B (text-space)	4	4	10	14	16	12	20	503	4
LLaDA-8B → Llama-3.2-3B (latent-space)									
plan (64, 128, 256 tokens) + executor : [‡]	2	2	4	5	5	5	6	14	2

数学推理任务（DART系列）上，DARM效果远远好于同结构纯ARM模型，证明隐空间在规划密集型任务上优势明显

实验结果

Model / Setting	ARC-E	ARC-C	DART-1	DART-2	DART-3	DART-4	DART-5	AIME24	MMLU
Accuracy									
DeepSeek-R1 (ARM only)	94.0	88.0	89.0	81.5	85.5	75.0	61.5	28.5	60.0
Qwen3-1.7B (ARM only)	92.0	86.0	89.5	86.0	83.0	73.0	49.0	8.5	68.5
Llama-3.2-3B (ARM only)	87.5	79.5	44.5	35.5	28.0	34.5	36.5	3.5	54.0
LLaDA-8B → Llama-3.2-3B (text-space)	90.5	82.5	53.5	43.0	35.5	30.0	27.0	0.0	52.5
LLaDA-8B → Llama-3.2-3B (latent-space)									
64 tokens plan	85.0	78.5	78.5	62.5	57.0	63.0	54.0	12.5	52.0
128 tokens plan	87.5	76.5	70.5	43.0	43.0	49.0	36.0	14.0	44.0
256 tokens plan	85.0	81.0	70.0	62.0	50.0	62.0	52.5	14.0	15.5
Number of Tokens in average									
DeepSeek-R1 (ARM only)	398	504	1420	1833	2076	2418	3068	3832	760
Qwen3-1.7B (ARM only)	282	397	1024	1669	2112	2550	3106	4036	984
Llama-3.2-3B (ARM only)	4	4	17	16	22	28	41	462	4
LLaDA-8B → Llama-3.2-3B (text-space)	4	4	10	14	16	12	20	503	4
LLaDA-8B → Llama-3.2-3B (latent-space)									
plan (64, 128, 256 tokens) + executor : [‡]	2	2	4	5	5	5	6	14	2

Latent-64 只用 ~70 tokens (64计划 + 6执行)
相比 DeepSeek-R1 节省 97.6% 的 token
准确率超越 Qwen3-1.7B (54.0% vs 49.0%)

03 实验

Model / Setting	ARC-E	ARC-C	DART-1	DART-2	DART-3	DART-4	DART-5	AIME24	MMLU
Accuracy									
DeepSeek-R1 (ARM only)	94.0	88.0	89.0	81.5	85.5	75.0	61.5	28.5	60.0
Qwen3-1.7B (ARM only)	92.0	86.0	89.5	86.0	83.0	73.0	49.0	8.5	68.5
Llama-3.2-3B (ARM only)	87.5	79.5	44.5	35.5	28.0	34.5	36.5	3.5	54.0
LLaDA-8B → Llama-3.2-3B (text-space)	90.5	82.5	53.5	43.0	35.5	30.0	27.0	0.0	52.5
LLaDA-8B → Llama-3.2-3B (latent-space)									
64 tokens plan	85.0	78.5	78.5	62.5	57.0	63.0	54.0	12.5	52.0
128 tokens plan	87.5	76.5	70.5	43.0	43.0	49.0	36.0	14.0	44.0
256 tokens plan	85.0	81.0	70.0	62.0	50.0	62.0	52.5	14.0	15.5
Number of Tokens in average									
DeepSeek-R1 (ARM only)	398	504	1420	1833	2076	2418	3068	3832	760
Qwen3-1.7B (ARM only)	282	397	1024	1669	2112	2550	3106	4036	984
Llama-3.2-3B (ARM only)	4	4	17	16	22	28	41	462	4
LLaDA-8B → Llama-3.2-3B (text-space)	4	4	10	14	16	12	20	503	4
LLaDA-8B → Llama-3.2-3B (latent-space)									
plan (64, 128, 256 tokens) + executor : [†]	2	2	4	5	5	5	6	14	2

- MMLU（大规模多任务语言理解） 上 Latent-Space 表现不佳
- 随着计划长度增加，性能急剧下降（256 token 时仅 15.5%）
- 说明：知识密集型任务不适合用隐空间传递长计划，不适合计划执行

Model / Setting	ARC-E	ARC-C	DART-1	DART-2	DART-3	DART-4	DART-5	AIME24	MMLU
Accuracy									
DeepSeek-R1 (ARM only)	94.0	88.0	89.0	81.5	85.5	75.0	61.5	28.5	60.0
Qwen3-1.7B (ARM only)	92.0	86.0	89.5	86.0	83.0	73.0	49.0	8.5	68.5
Llama-3.2-3B (ARM only)	87.5	79.5	44.5	35.5	28.0	34.5	36.5	3.5	54.0
LLaDA-8B → Llama-3.2-3B (text-space)	90.5	82.5	53.5	43.0	35.5	30.0	27.0	0.0	52.5
LLaDA-8B → Llama-3.2-3B (latent-space)									
64 tokens plan	85.0	78.5	78.5	62.5	57.0	63.0	54.0	12.5	52.0
128 tokens plan	87.5	76.5	70.5	43.0	43.0	49.0	36.0	14.0	44.0
256 tokens plan	85.0	81.0	70.0	62.0	50.0	62.0	52.5	14.0	15.5
Number of Tokens in average									
DeepSeek-R1 (ARM only)	398	504	1420	1833	2076	2418	3068	3832	760
Qwen3-1.7B (ARM only)	282	397	1024	1669	2112	2550	3106	4036	984
Llama-3.2-3B (ARM only)	4	4	17	16	22	28	41	462	4
LLaDA-8B → Llama-3.2-3B (text-space)	4	4	10	14	16	12	20	503	4
LLaDA-8B → Llama-3.2-3B (latent-space)									
plan (64, 128, 256 tokens) + executor : [†]	2	2	4	5	5	5	6	14	2

- MMLU（大规模多任务语言理解） 上 Latent-Space 表现不佳
- 随着计划长度增加，性能急剧下降（256 token 时仅 15.5%）
- 说明：知识密集型任务不适合用隐空间传递长计划，不适合计划执行

在不同任务上 Latent隐空间比文本交接有平均意义上的明显提升

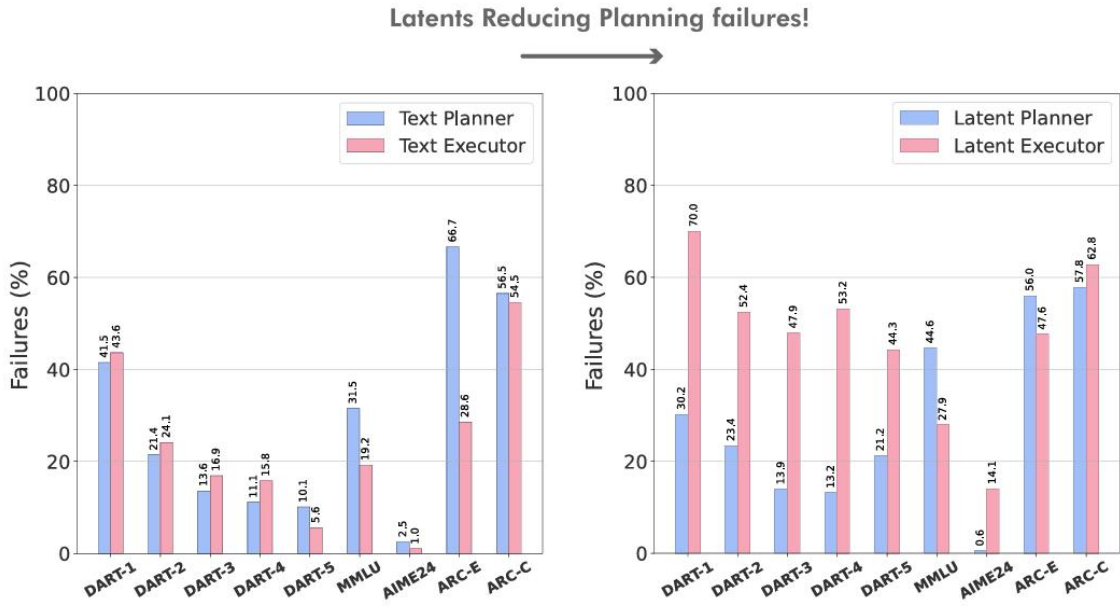
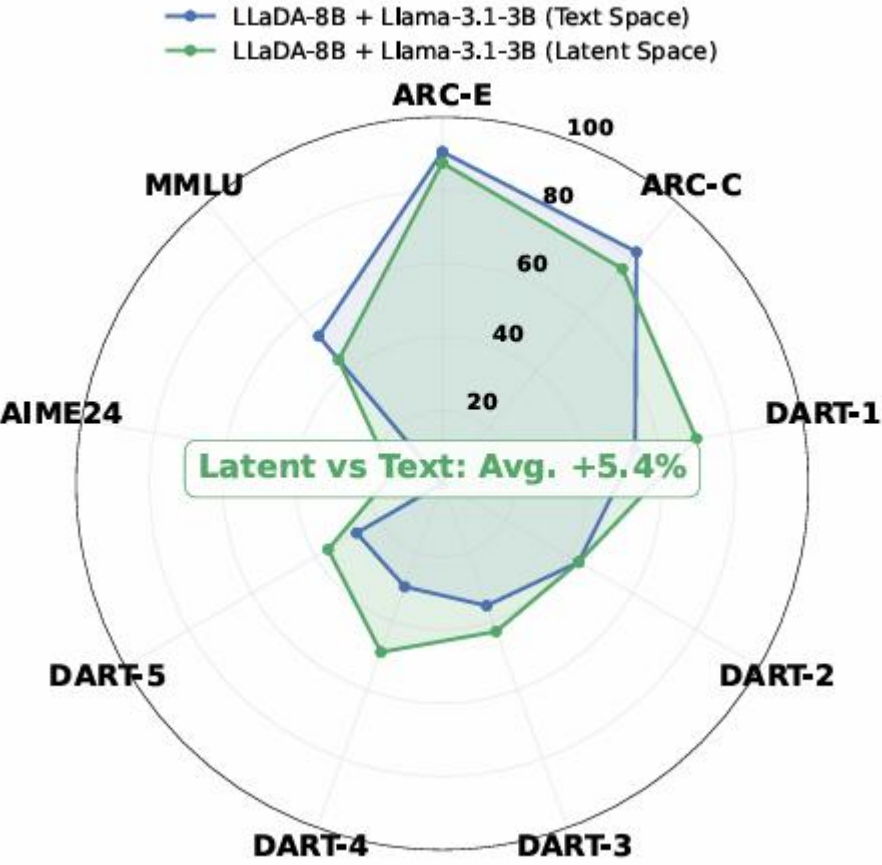
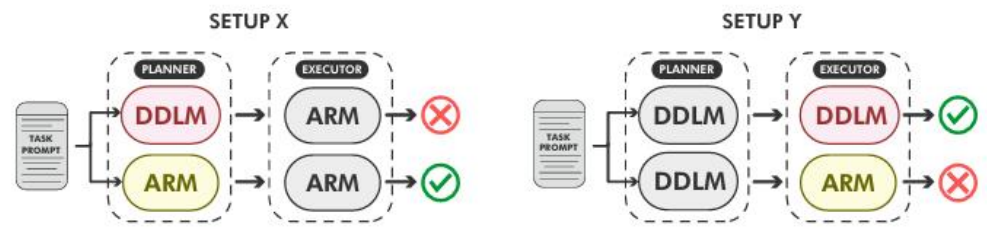


Figure 5: **Planner vs. executor failures in text- vs. latent-space collaboration.** Results for LLaDA-8B-Instruct and Llama-3.2-3B-Instruct. Latent-space collaboration substantially reduces planning errors compared to text-space.

如 Figure 5 所示，在文本空间协作下，大多数失败属于 Setup X，表明性能主要受限于文本解码导致的规划退化。相比之下，在 Latent-DARM 下，失败主要转移到 Setup Y，证明隐空间通信显著提高了规划保真度，而执行器成为主要的瓶颈。



04 小结

- Latent-DARM 通过一个可训练的隐空间投影器，让 DDLM 的规划"思想"直接流入 ARM，绕过文本解码的流畅性问题，在规划密集型任务上取得显著提升，同时将 token 效率提升近两个数量级，证明了隐空间是比文本更高效的智能体间通信媒介。
- 这说明agent之间或许可以通过向量直接进行交流 而不是译码为文本再编码回另一agent
- DDLM使得planner不需要用大量token完善文本输出正确性，在消费级平台上能实现极高的性价比
- 引发思考：自然语言是否agent间最好的交流手段？



Enabling Agents to Communicate Entirely in Latent Space “interlat”

Enabling Agents to Communicate Entirely in Latent Space

Zhuoyun Du^{1†*} Runze Wang^{2†} Huiyu Bai³ Zouying Cao⁴
Xiaoyong Zhu² Yu Cheng² Bo Zheng² Wei Chen¹ Haochao Ying^{1✉}

¹Zhejiang University ²Taobao & Tmail Group of Alibaba

³Nanyang Technological University ⁴Shanghai Jiao Tong University

{duzy, haochaoying}@zju.edu.cn, yunze.wrz@alibaba-inc.com

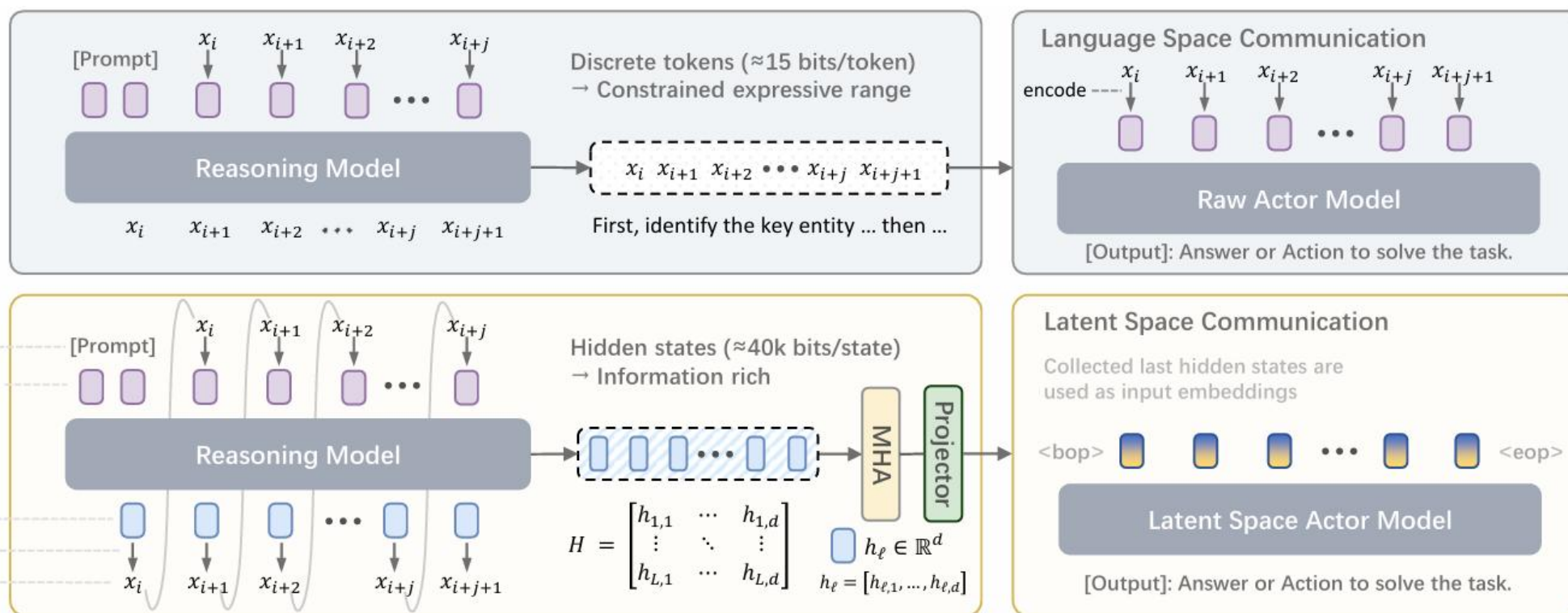
对Agent间隐状态通信的进一步
思考与探索

05 更好的“翻译模块”



北京大学 深圳研究生院
Peking University Shenzhen Graduate School

- 前文仅仅考虑了DDLm与ARM间的隐空间桥接问题，而Interlat则进一步拓展了这一范式。
- 浙江大学与阿里淘天联合构建的Interlat架构采用Transformer作为投影层，进一步地将方法拓展到了任意LLM、Agent间的协作



复合损失函数



为了使Interlat在同时满足多个通信要求，作者提出一个复合损失函数用于训练

$$\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{task}} + \lambda_S \mathcal{L}_{\text{sep}} + \lambda_A \mathcal{L}_{\text{align}}$$

其中

$$\mathcal{L}_{\text{task}} = -\frac{1}{|S|} \sum_{t \in S} \log p_{\theta}(y_t | C_t, H)$$

$\mathcal{L}_{\text{task}}$: 任务损失，确保模型生成准确的响应

S : 监督位置集合，即执行器输出中对应真实响应的那些位置

$|S|$: 监督位置的数量，用于归一化

t : 监督位置索引

y_t : 在位置 t 的真实词元 (ground-truth token)

C_t : 到位置 t 的解码器前缀，即模型已经生成的上下文

H : 匹配隐状态，即推理模型为当前任务生成的计划对应的最后一层隐状态

$p_{\theta}(y_t | C_t, H)$: 在给定前缀 C_t 和隐状态 H 的条件下，模型预测正确词元 y_t 的概率

作为经典的交叉熵损失函数，
确保执行器模型产生准确且连贯的响应

05 更好的“翻译模块”



$$\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{task}} + \lambda_S \mathcal{L}_{\text{sep}} + \lambda_A \mathcal{L}_{\text{align}}$$

$\mathcal{L}_{\text{sep}}$: 匹配损失

$$\mathcal{L}_{\text{sep}} = -\frac{1}{|S|} \sum_{t \in S} \text{JS} \left(p_{\theta}(\cdot | C_t, H), p_{\theta}(\cdot | C_t, \tilde{H}) \right)$$

其中, $\text{JS}(\cdot, \cdot)$ 为 Jensen-Shannon 散度, 衡量两个概率分布之间的差异

H : 匹配隐状态, 推理模型为**当前任务**生成的计划对应的最后一层隐状态

$\tilde{H}$: 不匹配隐状态, 从**不同任务**采样的隐状态

$p_{\theta}(\cdot | C_t, H)$: 给定前缀 C_t 和匹配隐状态 H 时, 模型在整个词表上的条件概率分布

$p_{\theta}(\cdot | C_t, \tilde{H})$: 给定前缀 C_t 和不匹配隐状态 $\tilde{H}$ 时, 模型在整个词表上的条件概率分布

训练目的: 通过最大化两个分布的差异, 使得模型具有辨别隐状态的能力

- 例如, 当agent1处理数学任务时, 同时收到数学家agent2与化学家agent3的隐状态输入, agent1可以从匹配隐状态 (即agent2的输出) 中获取有效的信息

05 更好的“翻译模块”



北京大学 深圳研究生院
Peking University Shenzhen Graduate School

$$\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{task}} + \lambda_S \mathcal{L}_{\text{sep}} + \lambda_A \mathcal{L}_{\text{align}}$$

$\mathcal{L}_{\text{align}}$: 对齐损失

$$\mathcal{L}_{\text{align}} = \frac{\beta}{|S|} \sum_{t \in S} \text{KL}(p_{\theta}(\cdot | H) \| p_{\text{plan}}(\cdot | P)) + \frac{\alpha}{|S|} \sum_{t \in S} (1 - \cos(\mathbf{z}_{\theta}(H), \mathbf{z}_{\text{plan}}(P)))$$

同一个任务，同一个模型，既生成隐状态 H （用于隐通信），也生成文本计划 P （作为监督信号）

限制隐空间输出尽可能与对应的真实文本进行匹配

KL散度控制了隐空间与文本空间的分布一致性，余弦相似度则控制两者的方向一致性

05 更好的“翻译模块”

$$\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{task}} + \lambda_S \mathcal{L}_{\text{sep}} + \lambda_A \mathcal{L}_{\text{align}}$$

*其中 λ_S 与 λ_A 对不同的模型有不同的取值

- 文章通过最小化 $\mathcal{L}_{\text{total}}$ 来对适配器（即Transformer投影层）中的权重进行优化
- 然而，完整的隐状态维度极大，存在大量冗余，因此一个用于生成压缩隐状态的LLM是有必要的
- 采用两阶段训练：**第一阶段**只训练通信适配器，让执行器学会利用隐状态；**第二阶段**冻结适配器和执行器，只训练推理模型生成压缩隐状态（其本质是一种追求高质量隐状态的蒸馏，用完整的隐状态蒸馏降维的隐状态）。这种设计隔离了变量，保证了训练的稳定性和模块的可复用性。

Ratio	Seen	Unseen	Time
Untrained (An instruction-tuned Model)			
128L	64.55 $\pm$ 2.26	60.25 $\pm$ 2.06	3.55s
64L	66.23 $\pm$ 1.95	61.53 $\pm$ 4.32	1.83s
32L	63.57 $\pm$ 2.01	60.18 $\pm$ 3.58	1.03s
16L	64.29 $\pm$ 1.34	60.00 $\pm$ 3.01	0.62s
8L	64.00 $\pm$ 2.18	57.46 $\pm$ 2.69	0.39s
Trained			
128L	68.10 $\pm$ 1.93	62.94 $\pm$ 2.03	2.25s
64L	<u>67.14</u> $\pm$ 1.56	<u>61.94</u> $\pm$ 2.13	1.16s
32L	66.90 $\pm$ 1.46	61.94 $\pm$ 2.56	0.60s
16L	66.43 $\pm$ 2.05	61.69 $\pm$ 2.56	0.33s
8L	66.43 $\pm$ 1.22	60.45 $\pm$ 2.23	0.20s

可以看出来，128步与8步隐状态对Seen任务与Unseen任务效果接近，而通信时间相差近十倍且经过训练后，正确率和时间都有明显优化

06 结果分析

Method	Qwen2.5-7B-Base				Qwen2.5-0.5B-Base				LLaMA3.1-8B			
	Seen	Steps	UnSeen	Steps	Seen	Steps	UnSeen	Steps	Seen	Steps	UnSeen	Steps
Interlat												
Ours	70.48	9.41/12.54	65.42	9.86/13.37	61.19	10.55/14.22	57.46	9.38/13.90	70.71	8.02/12.58	70.90	8.21/12.96
Text	64.29	8.76/12.77	62.44	9.79/13.63	54.52	9.50/14.28	47.26	9.70/15.13	62.86	7.91/12.94	60.82	8.14/13.21
No-Comm	62.14	10.19/13.90	62.19	10.23/13.92	50.48	8.23/14.06	44.03	9.10/15.20	63.57	8.35/12.59	58.40	9.47/13.85
Baselines												
CoT (full)	<u>67.14</u>	8.15/12.04	<u>64.93</u>	9.02/12.87	<u>57.86</u>	8.30/13.23	50.75	8.94/14.39	69.35	7.62/12.32	70.82	7.88/12.47
No-CoT	65.71	8.23/12.27	62.69	9.15/13.20	57.14	8.96/13.69	50.25	9.80/14.87	67.18	7.85/12.61	70.34	8.02/12.88
Variants												
CrossTask Noised	61.43	8.42/12.89	61.94	9.51/13.50	53.57	9.40/14.32	47.01	10.06/15.33	65.00	8.05/12.24	63.43	9.86/13.57
CovNoise-0.5×	64.29	8.54/12.63	60.95	8.71/13.12	53.33	8.80/14.03	46.77	9.64/15.16	64.29	8.10/12.50	65.68	9.47/12.77
CovNoise-1.0×	63.81	8.66/12.76	63.68	8.72/12.82	53.10	8.96/14.14	44.53	9.68/15.40	58.57	7.80/12.80	64.93	9.66/13.28
WhiteNoise	61.90	8.65/12.97	61.19	9.32/13.46	57.38	8.00/13.11	<u>57.21</u>	9.18/13.81	61.43	8.01/12.64	64.93	9.52/13.19
CovGauss-0 μ	60.00	8.79/13.27	61.94	9.59/13.55	13.81	11.25/18.79	<u>13.18</u>	12.93/19.07	57.86	8.04/13.08	66.42	9.51/13.03
CovGauss- μ	<u>65.71</u>	8.58/12.50	<u>64.93</u>	8.63/12.62	44.52	9.21/15.20	34.33	10.19/16.63	60.71	7.69/12.53	64.93	8.85/12.76
RandomRot	57.86	8.43/13.31	63.68	9.37/13.23	59.05	8.24/13.06	51.99	9.12/14.34	57.86	7.67/12.86	63.44	9.04/13.25
Cross family												
Qwen2LLaMA	70.95	8.47/12.01	71.39	9.21/13.05	–	–	–	–	–	–	–	–

- 在不同模型上，Interlat都取得了最好的效果，对比基线模型有明显提升
- 在变量扰动模式中CovNoise为对隐输出增加0.5/1.0 σ 协方差的高斯噪声（该噪声对高阶结构产生更大影响）；RandomRot则是对向量施加一个正交随机旋转矩阵（该矩阵保留原向量的均值与方差）。二者都对推理结果产生了较大的影响。说明模型确实依赖隐空间的高阶结构与几何方向信息

06 结果分析

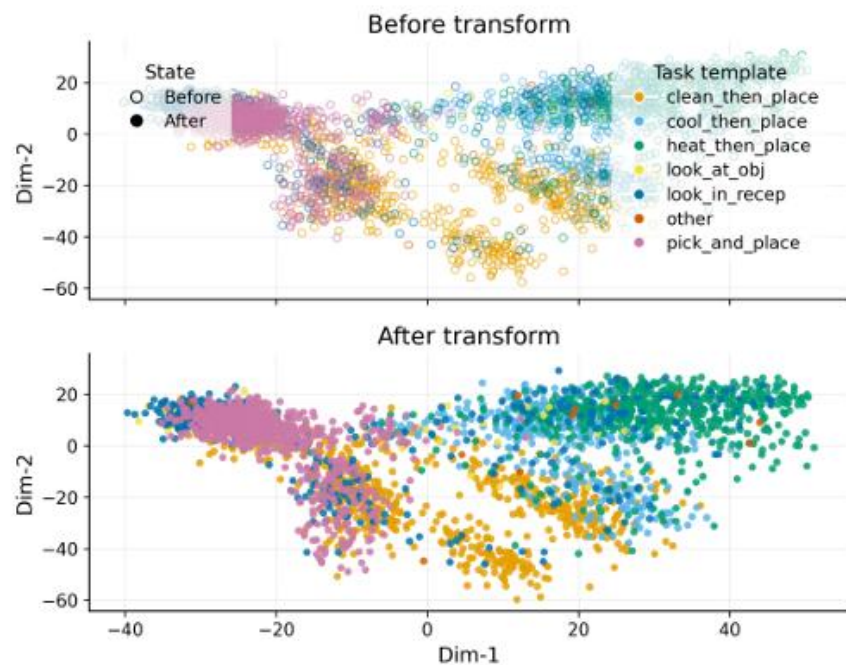


Figure 4: PCA visualization of latent communications, showing distinguishable task-specific structure in latent space both before and after the communication adapter.

隐空间2048维——2维PCA降维

图4的 PCA 可视化证明:

- 隐状态不是随机噪声，而是编码了任务特定的语义结构（不同任务模板形成可区分的簇）。通信适配器进一步增强了这种结构，让相同任务的隐状态更紧密聚集，不同任务的隐状态更明显分离。这为"隐空间通信有效"提供了直接的可视化证据，并解释了为什么执行器能根据隐状态区分不同任务。
- After transform 中，簇之间的分离更明显，类内聚集更紧密。这说明：通信适配器不仅"翻译"隐状态，还增强了任务相关的语义结构，让执行器更容易区分不同任务。
- 相似任务的簇在空间中相邻，如clean_then_place就与pick_and_place相邻，证明了在隐空间中任务模板的语义关系被保留

06 结果分析

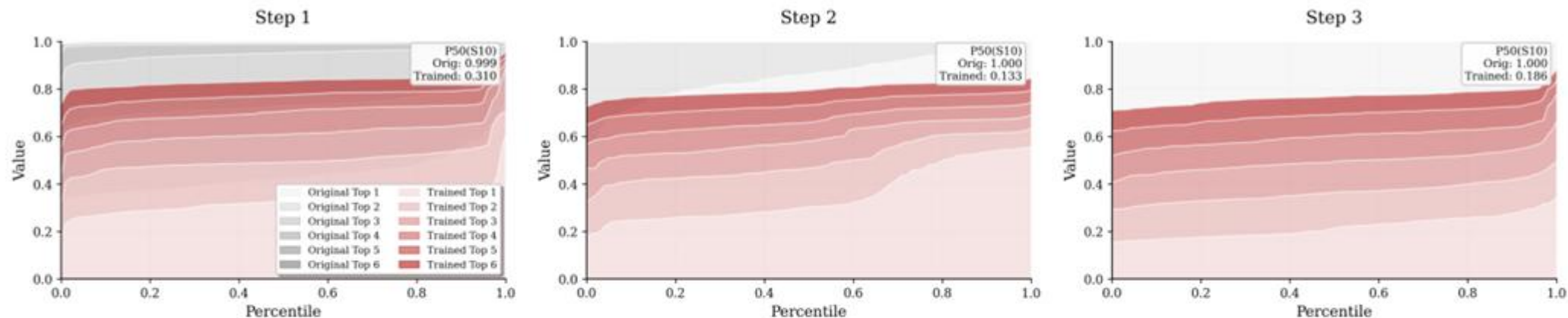


Figure 6: Analysis of parallelism in latent communication across the first six steps. The **red** denotes latents from the trained model, and the **gray** is the untrained model. Trained latents retain stable vertical gaps between successive Top- k bands and exhibit markedly lower $P_{50}(S_{10})$, indicating persistent parallelism, whereas the untrained latents progressively collapse toward Top-1. **See Appendix H for extended results.**

图6证明：未训练的压缩模型会快速将概率坍缩到 Top-1（过早确定单一答案），而训练后的 Interlat 模型能保持 Top-1 到 Top-6 曲线之间的稳定垂直间隔，维持多种合理的推理路径（并行性）。

- P50:对每个位置，计算前10个最可能词元的累积概率，取所有位置的中位数
P50显著降低，说明概率分布更宽、更均匀。这种并行性使 Agent 在面对复杂任务时更鲁棒，能根据环境反馈逐步排除错误选项，而不是一开始就锁定一个可能错误的答案。

消融实验

Actor	Seen	Unseen
Ours Full	70.48 ± 1.01	65.42 ± 0.87
w/o curri	33.10 ± 2.97	20.65 ± 2.15
w/o $\mathcal{L}_{\text{sep}}$	<u>58.81</u> ± 1.41	<u>60.70</u> ± 5.50
w/o $\mathcal{L}_{\text{align}}$	56.90 ± 1.41	53.98 ± 3.35
w/o adapter	4.05 ± 1.70	4.48 ± 1.31

Reasoning	Seen	Unseen
Ours Full	68.10 ± 1.93	<u>62.94</u> ± 2.03
w/o $\mathcal{L}_{\text{task}}$	65.71 ± 1.43	63.18 ± 3.47
w/o $\mathcal{L}_{\text{pref}}$	64.76 ± 2.97	60.20 ± 3.13
w/o $\mathcal{L}_{\text{geom}}$	64.05 ± 3.55	59.45 ± 3.01

配置	移除的组件	说明
Ours Full	无	完整方法（基线）
w/o curriculum	课程学习	直接从头学习解释隐状态
w/o L_sep	分离损失	不区分匹配/不匹配隐状态
w/o L_align	对齐损失	不对齐语言计划
w/o adapter	通信适配器	直接输入隐状态给执行器

- 单独移除任意组件对结果的影响程度都是十分明显的，尤其是适配器。这说明执行器训练的任何一个组件都是有必要的
- 第二阶段压缩训练的消融结果同样说明了复合压缩损失中的每一个损失函数都不可或缺

Interlat 证明了 Agent 可以不输出任何文本，完全通过隐状态进行通信，压缩到 8 步仍保持 94.3% 性能，加速 46 倍，且跨模型家族通信优于同家族。这一系列突破揭示了隐空间通信的三大核心优势：

- 第一，信息密度优势。自然语言通信的本质是将高维连续隐状态（约40k bits）下采样为离散token（约15 bits/token），这一过程必然造成信息损失。而隐空间通信直接传递完整表示，保留了推理过程中的多种可能性、备选路径和不确定性信息，使Agent能够维持并行推理而非过早坍缩。
- 第二，效率优势。完整隐状态序列（128步）耗时9.19秒，压缩到8步后仅需0.20秒，8步压缩后的性能仅下降4%，证明隐状态中存在大量可去除的冗余结构。
- 第三，泛化优势。Qwen生成的隐状态被LLaMA消费时，性能甚至优于同家族通信（70.95% vs 70.48%）。这证明隐状态编码的是模型无关的语义结构，而非特定模型的激活模式。**不同架构的模型可以共享同一个隐空间**，为异构多智能体协作开辟了全新可能。
- 更深层的意义：Interlat的工作挑战了“自然语言是智能体间最佳通信媒介”的根本假设。通过分离损失强迫模型主动区分任务相关信息、通过对齐损失防止失效模式、通过课程学习稳定训练，Interlat建立了完整的隐空间通信方法论。扰动实验进一步揭示，模型最依赖的是隐状态的方向/高阶结构，而非低阶统计量，这为理解隐空间的信息编码机制提供了重要线索。